Data Exploration with Python and Jupyter

Basic usage of the Pandas library to download a dataset, explore its contents, clean up missing or invalid data, filter the data according to different criteria, and plot visualizations of the data.

- Part 1: Python and Jupyter
- Part 2: Pandas with toy data
- **Part 3: Pandas with real data**

Press `Spacebar` *to go to the next slide (or* `?` *to see all navigation shortcuts)*

Loading [MathJax]/extensions/Safe.js

Let’s download some real data

For some reason, the London Fire Brigade provides a public spreadsheet of all animal rescue incidents since 2009:

https://data.london.gov.uk/dataset/animal-rescue-incidents-attended-by-lfb

They provide a link to the dataset in csv (comma-delimited) format

Loading [MathJax]/extensions/Safe.js

Let's download some real data

For some reason, the London Fire Brigade provides a public spreadsheet of all animal rescue incidents since 2009:

https://data.london.gov.uk/dataset/animal-rescue-incidents-attended-by-lfb

They provide a link to the dataset in csv (comma-delimited) format

In [1]:
```
# import the Pandas library & matplotlib for plotting

import pandas as pd
import matplotlib.pyplot as plt
```

Loading [MathJax]/extensions/Safe.js

Let’s download some real data

For some reason, the London Fire Brigade provides a public spreadsheet of all animal rescue incidents since 2009:

https://data.london.gov.uk/dataset/animal-rescue-incidents-attended-by-lfb

They provide a link to the dataset in csv (comma-delimited) format

In [1]:
```
# import the Pandas library & matplotlib for plotting

import pandas as pd
import matplotlib.pyplot as plt
```

In [2]:
```
# download a csv file with some data and convert it to a DataFrame
url = "https://data.london.gov.uk/download/animal-rescue-incidents-atte
df = pd.read_csv(url, encoding="unicode_escape")
```

Loading [MathJax]/extensions/Safe.js

Sidenote

You may be wondering what this `encoding='unicode_escape'` parameter is for, and how did I know it needed to be there?

Well, I first tried `pd.read_csv(url)` and got this:

```
---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
/tmp/ipykernel_514329/874975015.py in <module>
----> 1 pd.read_csv(url)

~/.pyenv/versions/3.9.4/lib/python3.9/site-packages/pandas/io/parsers.py in read_csv(filepath_or_buffer,
sep, delimiter, header, names, index_col, usecols, squeeze, prefix, mangle_dupe_cols, dtype, engine,
converters, true_values, false_values, skipinitialspace, skiprows, skipfooter, nrows, na_values,
keep_default_na, na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format, keep_date_col,
date_parser, dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal, lineterminator,
quotechar, quoting, doublequote, escapechar, comment, encoding, dialect, error_bad_lines, warn_bad_lines,
delim_whitespace, low_memory, memory_map, float_precision, storage_options)
    608     kwds.update(kwds_defaults)
    609
--> 610     return _read(filepath_or_buffer, kwds)
    611
    612

~/.pyenv/versions/3.9.4/lib/python3.9/site-packages/pandas/io/parsers.py in _read(filepath_or_buffer,
kwds)
    460
    461     # Create the parser.
--> 462     parser = TextFileReader(filepath_or_buffer, **kwds)
    463
    464     if chunksize or iterator:

~/.pyenv/versions/3.9.4/lib/python3.9/site-packages/pandas/io/parsers.py in __init__(self, f, engine,
**kwds)
```

Loading [MathJax]/extensions/Safe.js

Suggested workflow / philosophy

- you want to do something
    - if you know / have a guess which function to use, look at its docstring: `?function_name`
    - if you don't have any idea what to try, google `how do I ... in pandas`
    - if in doubt, just try something!
- if you get an error, copy & paste the last bit into google (along with `funtion_name` and/or `pandas`)
    - don't be intimidated by the long and apparently nonsensical error messages
    - almost certainly someone else has had this exact problem
    - almost certainly the solution is waiting for you
- look for a stackoverflow answer with many up-votes
    - ignore the green tick, this just means the person asking the question liked the answer
    - typically an answer with many up-votes is a better option
    - more recent answers can also be better: sometimes a library has changed since an older answer was written

Loading [MathJax]/extensions/Safe.js

Display the DataFrame

Loading [MathJax]/extensions/Safe.js

Loading [MathJax]/extensions/Safe.js

Display the DataFrame

In [3]:
```
df
```

Out[3]:

	IncidentNumber	DateTimeOfCall	CalYear	FinYear	TypeOfIncident	Pum
0	139091	01/01/2009 03:01	2009	2008/09	Special Service	
1	275091	01/01/2009 08:51	2009	2008/09	Special Service	
2	2075091	04/01/2009 10:07	2009	2008/09	Special Service	
3	2872091	05/01/2009 12:27	2009	2008/09	Special Service	
4	3553091	06/01/2009 15:23	2009	2008/09	Special Service	
...	...	...	...	...	...	
8485	077969-31052022	31/05/2022 06:18	2022	2022/23	Special Service	

Loading [MathJax]/extensions/Safe.js

	IncidentNumber	DateTimeOfCall	CalYear	FinYear	TypeOfIncident	Pum
8486	078069-31052022	31/05/2022 10:55	2022	2022/23	Special Service	
8487	078094-31052022	31/05/2022 11:50	2022	2022/23	Special Service	
8488	078340-31052022	31/05/2022 20:27	2022	2022/23	Special Service	
8489	078347-31052022	31/05/2022 20:35	2022	2022/23	Special Service	

8490 rows × 31 columns

Loading [MathJax]/extensions/Safe.js

Column data types

Loading [MathJax]/extensions/Safe.js

Loading [MathJax]/extensions/Safe.js

Column data types

In [4]:
```
df.dtypes
```

Out[4]:
```
IncidentNumber                 object
DateTimeOfCall                 object
CalYear                         int64
FinYear                        object
TypeOfIncident                 object
PumpCount                     float64
PumpHoursTotal                float64
HourlyNotionalCost(£)           int64
IncidentNotionalCost(£)       float64
FinalDescription               object
AnimalGroupParent              object
OriginofCall                   object
PropertyType                   object
PropertyCategory               object
SpecialServiceTypeCategory     object
SpecialServiceType             object
WardCode                       object
Ward                           object
BoroughCode                    object
Borough                        object
```

Loading [MathJax]/extensions/Safe.js

```
StnGroundName                 object
UPRN                         float64
Street                        object
USRN                         float64
PostcodeDistrict              object
Easting_m                    float64
Northing_m                   float64
Easting_rounded                int64
Northing_rounded               int64
Latitude                     float64
Longitude                    float64
dtype: object
```

Loading [MathJax]/extensions/Safe.js

Convert DateTimeOfCall to a date-time

Loading [MathJax]/extensions/Safe.js

Convert DateTimeOfCall to a date-time

In [5]:
```
df["DateTimeOfCall"].head()
```

Out[5]:
```
0    01/01/2009 03:01
1    01/01/2009 08:51
2    04/01/2009 10:07
3    05/01/2009 12:27
4    06/01/2009 15:23
Name: DateTimeOfCall, dtype: object
```

Loading [MathJax]/extensions/Safe.js

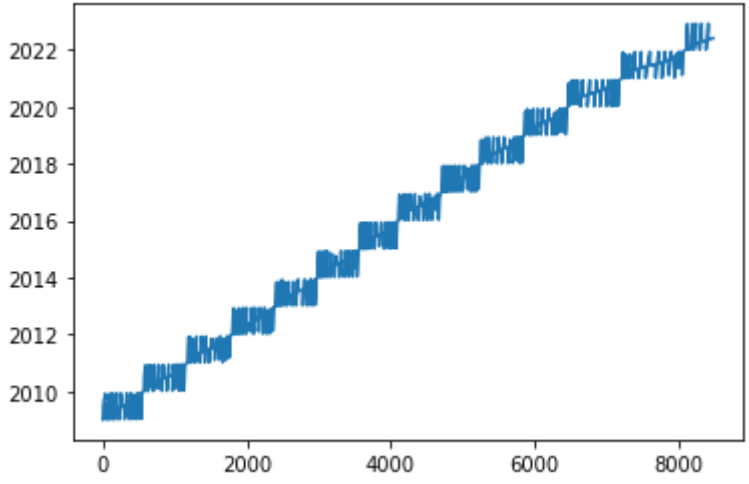
In [6]:
```
# this looks like what we want..
pd.to_datetime(df["DateTimeOfCall"]).head()
```

Out[6]:
```
0   2009-01-01 03:01:00
1   2009-01-01 08:51:00
2   2009-04-01 10:07:00
3   2009-05-01 12:27:00
4   2009-06-01 15:23:00
Name: DateTimeOfCall, dtype: datetime64[ns]
```

Loading [MathJax]/extensions/Safe.js

In [7]:
```
# ..but which number is the month and which is the day?
# how can we check if what we just did makes sense?
pd.to_datetime(df["DateTimeOfCall"]).plot()
# should be a single monotonically increasing line: looks like days/mon
```

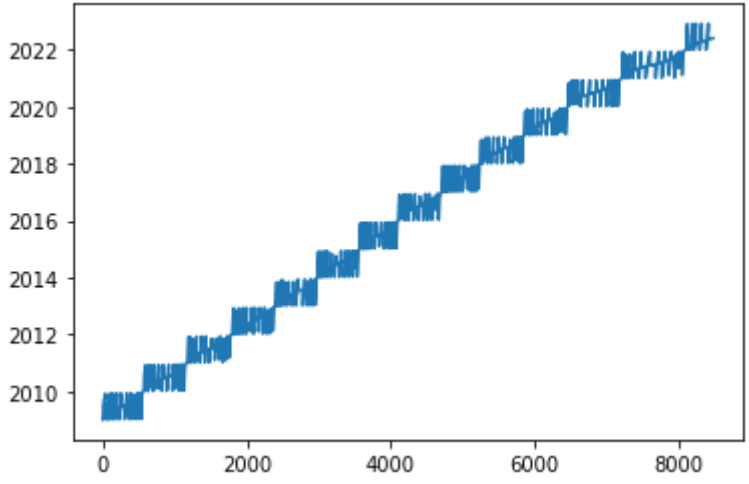
Out[7]:
```
<AxesSubplot:>
```



Loading [MathJax]/extensions/Safe.js

In [7]:
```
# ..but which number is the month and which is the day?
# how can we check if what we just did makes sense?
pd.to_datetime(df["DateTimeOfCall"]).plot()
# should be a single monotonically increasing line: looks like days/mon
```

Out[7]:
```
<AxesSubplot:>
```



In [8]:
```
# check the docs to see what options are available:
?pd.to_datetime
```

Loading [MathJax]/extensions/Safe.js

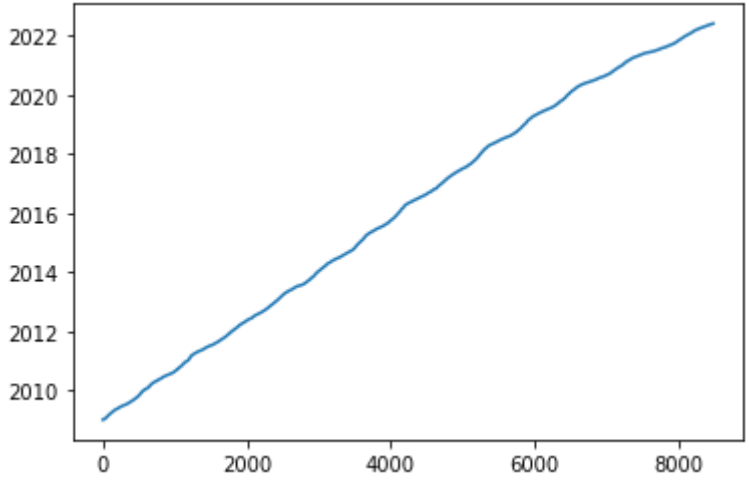
In [9]:
```
# this looks better
pd.to_datetime(df["DateTimeOfCall"], dayfirst=True).head()
```

Out[9]:
```
0   2009-01-01 03:01:00
1   2009-01-01 08:51:00
2   2009-01-04 10:07:00
3   2009-01-05 12:27:00
4   2009-01-06 15:23:00
Name: DateTimeOfCall, dtype: datetime64[ns]
```

Loading [MathJax]/extensions/Safe.js

In [10]:
```
# consistency check: plot datetime vs index, should increase monotonica
pd.to_datetime(df["DateTimeOfCall"], dayfirst=True).plot()
```

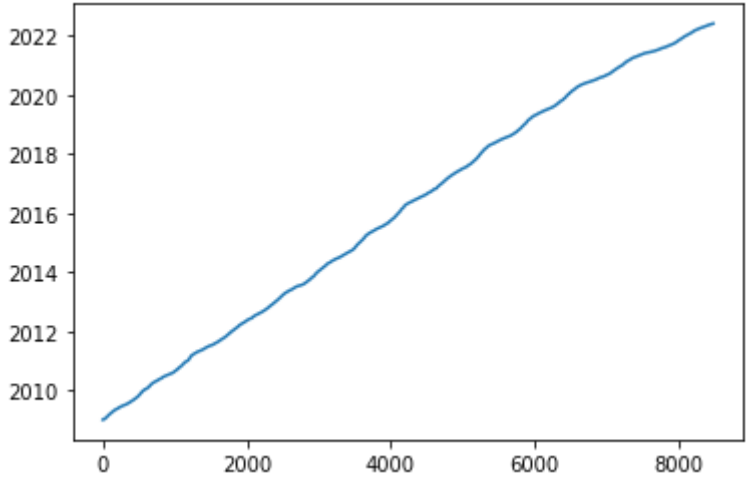
Out[10]:
```
<AxesSubplot:>
```



Loading [MathJax]/extensions/Safe.js

In [10]:
```
# consistency check: plot datetime vs index, should increase monotonica
pd.to_datetime(df["DateTimeOfCall"], dayfirst=True).plot()
```

Out[10]:
```
<AxesSubplot:>
```



In [11]:
```
# replace DateTimeOfCall column in dataframe with this one
df["DateTimeOfCall"] = pd.to_datetime(df["DateTimeOfCall"], dayfirst=Tr
```

Loading [MathJax]/extensions/Safe.js

Use the datetime as the index

Loading [MathJax]/extensions/Safe.js

Loading [MathJax]/extensions/Safe.js

Use the datetime as the index

In [12]:
```
df.set_index("DateTimeOfCall", inplace=True)
```

Loading [MathJax]/extensions/Safe.js

Loading [MathJax]/extensions/Safe.js

Use the datetime as the index

In [12]:
```
df.set_index("DateTimeOfCall", inplace=True)
```

In [13]:
```
df
```

Out[13]:

	IncidentNumber	CalYear	FinYear	TypeOfIncident	PumpCount
DateTimeOfCall					
2009-01-01 03:01:00	139091	2009	2008/09	Special Service	1.0
2009-01-01 08:51:00	275091	2009	2008/09	Special Service	1.0
2009-01-04 10:07:00	2075091	2009	2008/09	Special Service	1.0
2009-01-05 12:27:00	2872091	2009	2008/09	Special Service	1.0

Loading [MathJax]/extensions/Safe.js

	IncidentNumber	CalYear	FinYear	TypeOfIncident	PumpCount
DateTimeOfCall					
2009-01-06 15:23:00	3553091	2009	2008/09	Special Service	1.0
...	...	...	...	...	...
2022-05-31 06:18:00	077969-31052022	2022	2022/23	Special Service	1.0
2022-05-31 10:55:00	078069-31052022	2022	2022/23	Special Service	1.0
2022-05-31 11:50:00	078094-31052022	2022	2022/23	Special Service	1.0
2022-05-31 20:27:00	078340-31052022	2022	2022/23	Special Service	1.0
2022-05-31 20:35:00	078347-31052022	2022	2022/23	Special Service	1.0

8490 rows × 30 columns

Loading [MathJax]/extensions/Safe.js

In [14]:
```
# can now use datetime to select rows: here is jan 2021
df.loc["2021-01-01":"2021-01-31", "FinalDescription"]
```

Out[14]:
```
DateTimeOfCall
2021-01-01 12:09:00        KITTEN STUCK UP TREE  AL REQUESTED F
ROM SCENE
2021-01-01 14:06:00
Redacted
2021-01-03 18:40:00                CAT WITH LEG TRAPPED IN BATH
PLUGHOLE
2021-01-04 13:39:00
Redacted
2021-01-06 10:22:00
Redacted
2021-01-06 13:09:00    CAT IN DISTRESS ON ROOF - ADDITIONAL APP
LIANCE...
2021-01-06 20:35:00        DOG TRAPPED IN FOX HOLE  - MEET AT C
LUB HOUSE
2021-01-07 23:50:00                   KITTEN STUCK BETWEEN WALL
AND ROOF
2021-01-09 08:01:00                                  DOG STUCK
IN TRENCH
2021-01-10 19:27:00
Redacted
2021-01-12 11:39:00
Redacted
2021-01-12 22:38:00                                 CAT TRAPPED
IN DITCH
2021-01-16 18:05:00                            DOG TRAPPED IN POR
TER CABIN
```

Loading [MathJax]/extensions/Safe.js

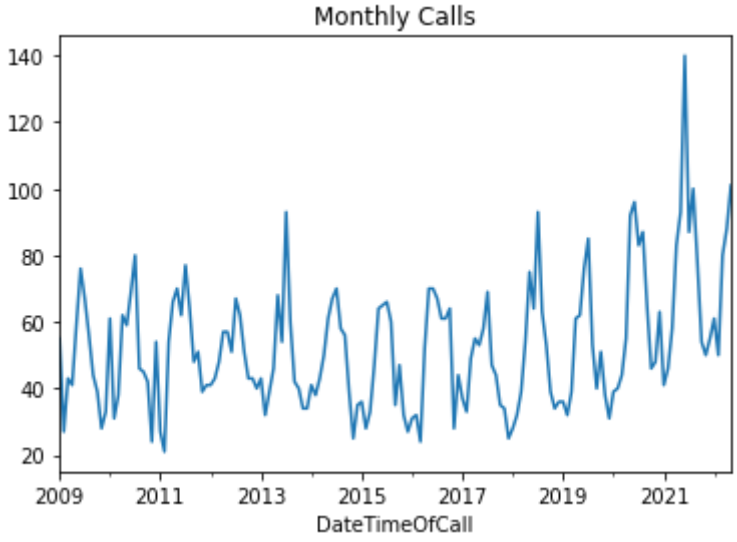
2021-01-17 16:09:00 DOG TRAPPED IN WAREHOUSE AREA - CALLER B
ELIEVE...
2021-01-17 17:09:00 BIRD TRAPPED IN NETTING CALLER WILL
MEET YOU
2021-01-18 15:17:00 CAT STUCK IN TREE BEING ATTACKED
BY CROWS
2021-01-18 17:06:00 ASSIST RSPCA - SMALL ANIMAL RESUE - BIRD
ENTAN...
2021-01-19 18:28:00 CAT TRAPPED BEHIND
CUPBOARD
2021-01-19 20:24:00
Redacted
2021-01-19 20:36:00 RUNNING CALL A
T ON ROOF
2021-01-20 09:35:00 CAT STUCK BETWEEN TREE
BRANCHES
2021-01-21 13:15:00 SWAN TRAPPED I
N NETTING
2021-01-21 18:23:00 CAT TRAPPED I
N CHIMNEY
2021-01-22 14:22:00 CAT TRAPPED BETWEEN WALL
AND FENCE
2021-01-23 10:18:00 CAT TRAPPED I
N CHIMNEY
2021-01-23 15:43:00 CAT TRAPPED BETW
EEN WALLS
2021-01-23 17:16:00
Redacted
2021-01-25 12:02:00 ASSIST RSPCA WITH FOX STUCK DOW
N CULVERT
2021-01-26 13:42:00 DOG STUCK IN RAILINGS - CALLER WILL

Loading [MathJax]/extensions/Safe.js

```
MEET YOU
2021-01-26 18:21:00
Redacted
2021-01-26 22:44:00    BIRDS TRAPPED IN BASKETBALL COURT CALLER
IS ON...
2021-01-26 23:35:00             FOX TRAPPED IN FENCE IN ALLEYWA
Y NEXT TO
2021-01-27 09:18:00    CAT STUCK IN TREE - ATTENDED YESTERDAY A
ND ADV...
2021-01-27 10:12:00    BIRD TRAPPED BY LEG IN A TREE - RSPCA IN
ATTEN...
2021-01-27 15:22:00                          CAT UP TREE   ASS
IST RSPCA
2021-01-29 10:47:00                TRAPPED FOX IN FENCE  IN RE
AR GARDEN
2021-01-30 14:53:00                                  CAT STUCK U
NDER SHED
2021-01-30 15:28:00              BIRD CAUGHT IN NETTING - RSPCA
ON SCENE
2021-01-30 17:54:00                                 DOG TRAPPED
UNDER CAR
2021-01-31 12:53:00                  CAT STUCK UP TREE - RSPCA
ON SCENE
2021-01-31 13:48:00           INJURED CAT STUCK IN GREEN AREA A
T REAR OF
Name: FinalDescription, dtype: object
```

Loading [MathJax]/extensions/Safe.js

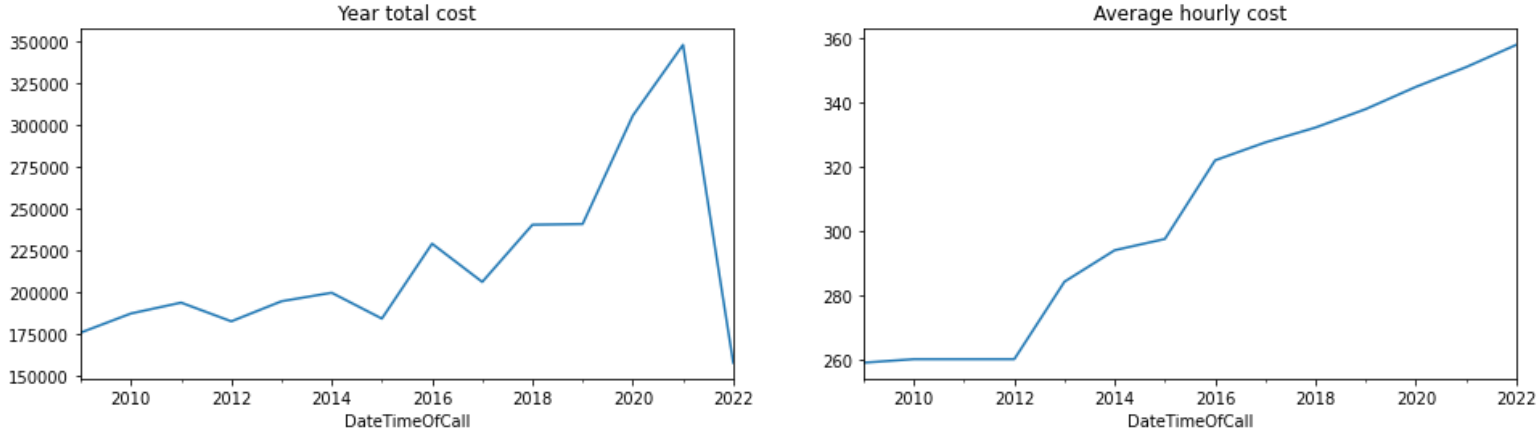
In [15]:
```
# resample the timeseries by month and count incidents
df.resample("M")["IncidentNumber"].count().plot(title="Monthly Calls")
# see https://pandas.pydata.org/docs/user_guide/timeseries.html#timeser
plt.show()
```



Loading [MathJax]/extensions/Safe.js

In [16]:

```
# resample by year, sum total costs, average hourly costs
fig, axs = plt.subplots(figsize=(16, 4), ncols=2)
df.resample("Y")["IncidentNotionalCost(£)"].sum().plot(
    title="Year total cost", ax=axs[0]
)
df.resample("Y")["HourlyNotionalCost(£)"].mean().plot(
    title="Average hourly cost", ax=axs[1]
)
plt.show()
```



Loading [MathJax]/extensions/Safe.js

Missing data

Different strategies for dealing with missing data:

- Ignore the issue
    - some things may break / not work as expected
- Remove rows/columns with missing data
    - remove all rows with missing data: `df.dropna(axis=0)`
    - remove all columns with missing data: `df.dropna(axis=1)`
- Guess (impute) missing data
    - replace all missing entries with a value: `df.fillna(1)`
    - replace missing entries with mean for that column
      `df.fillna(df.mean())`
    - replace each missing entry with previous valid entry:
      `df.fillna(method="pad")`
    - replace missing by interpolating between valid entries:
      `df.interpolate()`

Loading [MathJax]/extensions/Safe.js

In [17]:
```
# count missing entries for each column
df.isna().sum()
```

Out[17]:
```
IncidentNumber                    0
CalYear                           0
FinYear                           0
TypeOfIncident                    0
PumpCount                        57
PumpHoursTotal                   58
HourlyNotionalCost(£)             0
IncidentNotionalCost(£)          58
FinalDescription                  5
AnimalGroupParent                 0
OriginofCall                      0
PropertyType                      0
PropertyCategory                  0
SpecialServiceTypeCategory        0
SpecialServiceType                0
WardCode                         12
Ward                             12
BoroughCode                      11
Borough                          11
StnGroundName                     0
UPRN                           5402
Street                            0
USRN                           1156
PostcodeDistrict                  0
Easting_m                      4383
Northing_m                     4383
Easting_rounded                   0
```

Loading [MathJax]/extensions/Safe.js

```
Northing_rounded                 0
Latitude                      4383
Longitude                     4383
dtype: int64
```

Loading [MathJax]/extensions/Safe.js

In [18]:

```
# If PumpCount is missing, typically so is PumpHoursTotal
# 55 rows are missing at least one of these
pump_missing = df["PumpCount"].isna() | df["PumpHoursTotal"].isna()
print(pump_missing.sum())
```

```
58
```

Loading [MathJax]/extensions/Safe.js

In [18]:
```
# If PumpCount is missing, typically so is PumpHoursTotal
# 55 rows are missing at least one of these
pump_missing = df["PumpCount"].isna() | df["PumpHoursTotal"].isna()
print(pump_missing.sum())
```

```
58
```

In [19]:
```
# so we could choose to drop these rows
df1 = df.drop(df.loc[pump_missing == True].index)
# here we made a new dataset df1 with these rows dropped
# to drop the rows from the original dataset df, could do:
#
# df = df.drop(df.loc[pump_missing == True].index)
#
# or:
#
# df.drop(df.loc[pump_missing == True].index, inplace=True)
#
print(len(df1))
```

```
8432
```

Loading [MathJax]/extensions/Safe.js

In [18]:
```
# If PumpCount is missing, typically so is PumpHoursTotal
# 55 rows are missing at least one of these
pump_missing = df["PumpCount"].isna() | df["PumpHoursTotal"].isna()
print(pump_missing.sum())
```

```
58
```

In [19]:
```
# so we could choose to drop these rows
df1 = df.drop(df.loc[pump_missing == True].index)
# here we made a new dataset df1 with these rows dropped
# to drop the rows from the original dataset df, could do:
#
# df = df.drop(df.loc[pump_missing == True].index)
#
# or:
#
# df.drop(df.loc[pump_missing == True].index, inplace=True)
#
print(len(df1))
```

```
8432
```

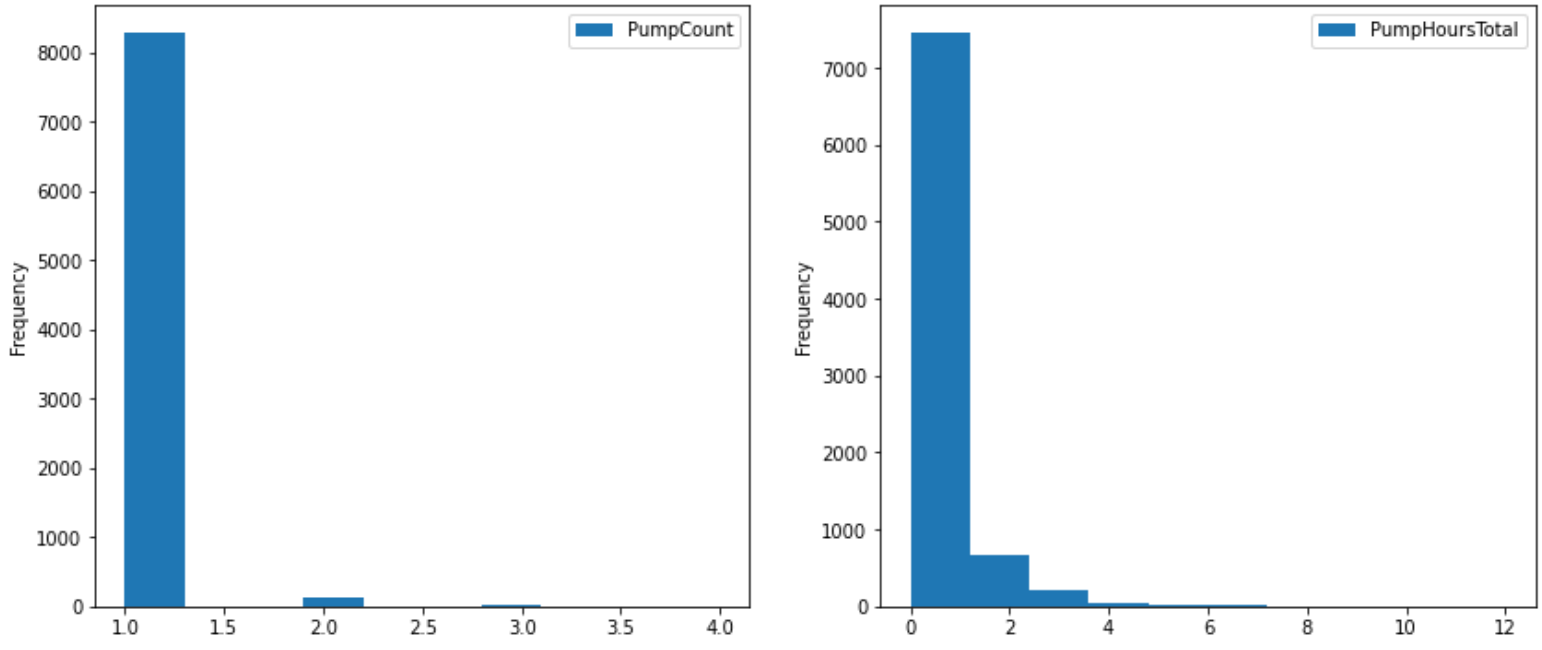
In [20]:
```
# another equivalent way to do this
df2 = df.dropna(subset=["PumpCount", "PumpHoursTotal"])
print(len(df2))
```

```
8432
```

Loading [MathJax]/extensions/Safe.js

In [21]:
```
# but if we drop them, we lose valid data from other columns
# let's look at the distribution of values:
fig, axs = plt.subplots(1, 2, figsize=(14, 6))
df.plot.hist(y="PumpCount", ax=axs[0])
df.plot.hist(y="PumpHoursTotal", ax=axs[1])
plt.plot()
```

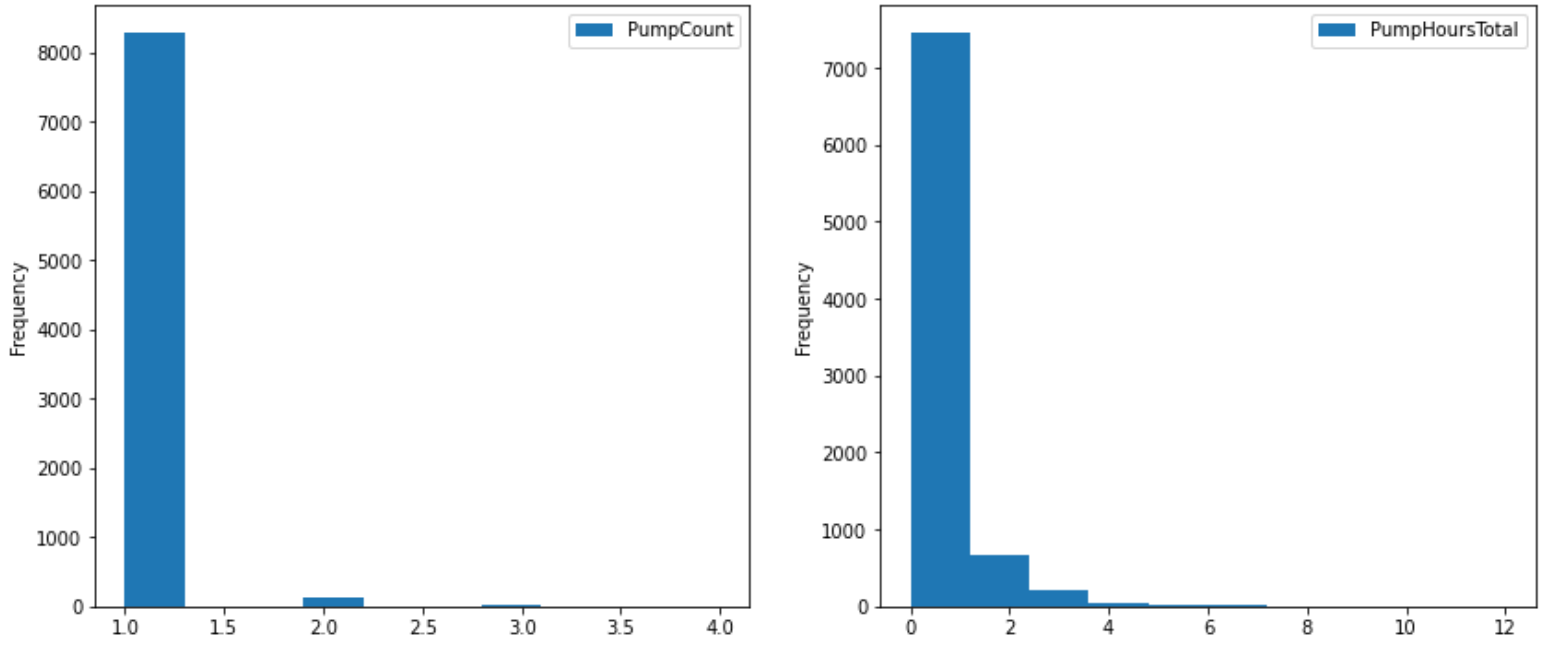
Out[21]:
```
[]
```



Loading [MathJax]/extensions/Safe.js

In [21]:
```
# but if we drop them, we lose valid data from other columns
# let's look at the distribution of values:
fig, axs = plt.subplots(1, 2, figsize=(14, 6))
df.plot.hist(y="PumpCount", ax=axs[0])
df.plot.hist(y="PumpHoursTotal", ax=axs[1])
plt.plot()
```

Out[21]:
```
[]
```



In [22]:
```
# looks like it would be better to replace missing PumpCount and PumpHc
?df.fillna
df.fillna({"PumpCount": 1, "PumpHoursTotal": 1}, inplace=True)
```

Loading [MathJax]/extensions/Safe.js

In [23]:
```
df.isna().sum()
```

Out[23]:
```
IncidentNumber                    0
CalYear                           0
FinYear                           0
TypeOfIncident                    0
PumpCount                         0
PumpHoursTotal                    0
HourlyNotionalCost(£)             0
IncidentNotionalCost(£)          58
FinalDescription                  5
AnimalGroupParent                 0
OriginofCall                      0
PropertyType                      0
PropertyCategory                  0
SpecialServiceTypeCategory        0
SpecialServiceType                0
WardCode                         12
Ward                             12
BoroughCode                      11
Borough                          11
StnGroundName                     0
UPRN                           5402
Street                            0
USRN                           1156
PostcodeDistrict                  0
Easting_m                      4383
Northing_m                     4383
Easting_rounded                   0
Northing_rounded                  0
```

Loading [MathJax]/extensions/Safe.js

```
Latitude                        4383
Longitude                       4383
dtype: int64
```

Loading [MathJax]/extensions/Safe.js

Count the unique entries in each column

Loading [MathJax]/extensions/Safe.js

Loading [MathJax]/extensions/Safe.js

Count the unique entries in each column

In [24]:
```
df.nunique().sort_values()
```

Out[24]:
```
TypeOfIncident                  1
PumpCount                       4
SpecialServiceTypeCategory      4
PropertyCategory                7
OriginofCall                    8
PumpHoursTotal                 12
HourlyNotionalCost(£)          12
CalYear                        14
FinYear                        15
SpecialServiceType             24
AnimalGroupParent              28
BoroughCode                    37
Borough                        70
IncidentNotionalCost(£)        76
StnGroundName                 107
PropertyType                  184
```

Loading [MathJax]/extensions/Safe.js

```
PostcodeDistrict          275
Northing_rounded          423
Easting_rounded           526
WardCode                  749
Ward                     1285
UPRN                     1698
Northing_m               3742
Easting_m                3798
Latitude                 4037
Longitude                4038
FinalDescription         5151
USRN                     5710
Street                   6438
IncidentNumber           8490
dtype: int64
```

Loading [MathJax]/extensions/Safe.js

In [25]:
```
# this column is always the same:
df["TypeOfIncident"].unique()
```

Out[25]:
```
array(['Special Service'], dtype=object)
```

Loading [MathJax]/extensions/Safe.js

In [25]:
```
# this column is always the same:
df["TypeOfIncident"].unique()
```

Out[25]:
```
array(['Special Service'], dtype=object)
```

In [26]:
```
# so not interesting - we can drop it from our dataframe:
df.drop("TypeOfIncident", axis=1, inplace=True)
```

Loading [MathJax]/extensions/Safe.js

In [27]:
```
# "cat" and "Cat" are treated as different animals here:
df["AnimalGroupParent"].unique()
```

Out[27]:
```
array(['Dog', 'Fox', 'Horse', 'Rabbit',
       'Unknown - Heavy Livestock Animal', 'Squirrel', 'Cat',
'Bird',
       'Unknown - Domestic Animal Or Pet', 'Sheep', 'Deer',
       'Unknown - Wild Animal', 'Snake', 'Lizard', 'Hedgehog',
'cat',
       'Hamster', 'Lamb', 'Fish', 'Bull', 'Cow', 'Ferret', 'Bud
gie',
       'Unknown - Animal rescue from water - Farm animal', 'Pig
eon',
       'Goat', 'Tortoise',
       'Unknown - Animal rescue from below ground - Farm anima
l'],
      dtype=object)
```

Loading [MathJax]/extensions/Safe.js

In [27]:
```
# "cat" and "Cat" are treated as different animals here:
df["AnimalGroupParent"].unique()
```

Out[27]:
```
array(['Dog', 'Fox', 'Horse', 'Rabbit',
       'Unknown - Heavy Livestock Animal', 'Squirrel', 'Cat',
'Bird',
       'Unknown - Domestic Animal Or Pet', 'Sheep', 'Deer',
       'Unknown - Wild Animal', 'Snake', 'Lizard', 'Hedgehog',
'cat',
       'Hamster', 'Lamb', 'Fish', 'Bull', 'Cow', 'Ferret', 'Bud
gie',
       'Unknown - Animal rescue from water - Farm animal', 'Pig
eon',
       'Goat', 'Tortoise',
       'Unknown - Animal rescue from below ground - Farm anima
l'],
      dtype=object)
```

In [28]:
```
# select rows where AnimalGroupParent is "cat", replace with "Cat"
df.loc[df["AnimalGroupParent"] == "cat", "AnimalGroupParent"] = "Cat"
```

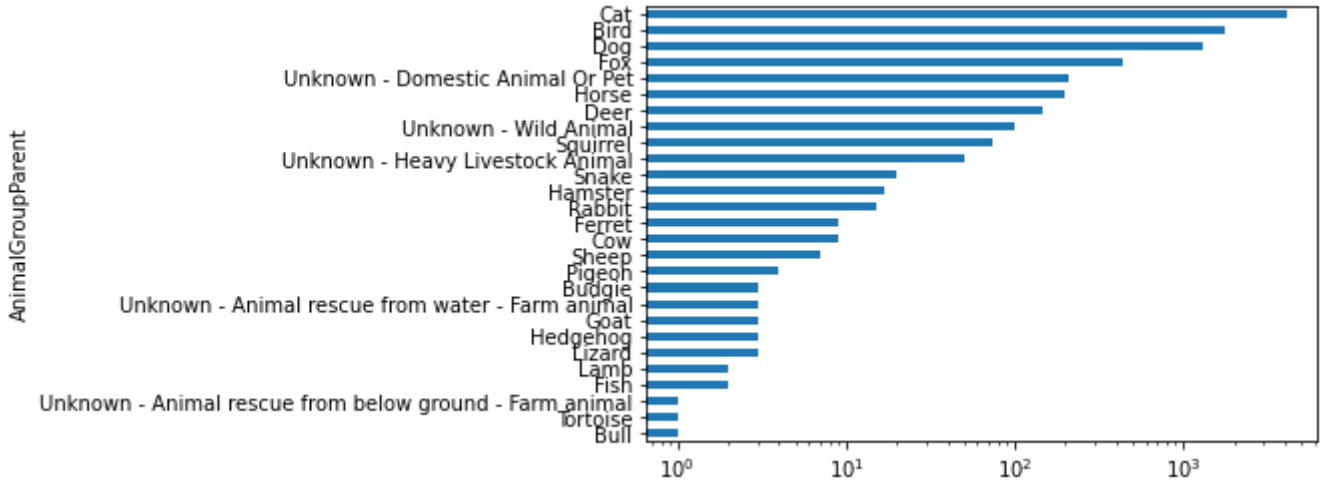
Loading [MathJax]/extensions/Safe.js

In [29]:
```
df["AnimalGroupParent"].unique()
```

Out[29]:
```
array(['Dog', 'Fox', 'Horse', 'Rabbit',
       'Unknown - Heavy Livestock Animal', 'Squirrel', 'Cat',
'Bird',
       'Unknown - Domestic Animal Or Pet', 'Sheep', 'Deer',
       'Unknown - Wild Animal', 'Snake', 'Lizard', 'Hedgehog',
'Hamster',
       'Lamb', 'Fish', 'Bull', 'Cow', 'Ferret', 'Budgie',
       'Unknown - Animal rescue from water - Farm animal', 'Pig
eon',
       'Goat', 'Tortoise',
       'Unknown - Animal rescue from below ground - Farm anima
l'],
      dtype=object)
```

Loading [MathJax]/extensions/Safe.js

In [30]:
```
df.groupby("AnimalGroupParent")["IncidentNumber"].count().sort_values()
    logx=True
)
plt.show()
```



Loading [MathJax]/extensions/Safe.js

In [31]:

```
# apparently different hourly costs
# does it depend on the type of event? or does it just increase over ti
df["HourlyNotionalCost(£)"].unique()
```

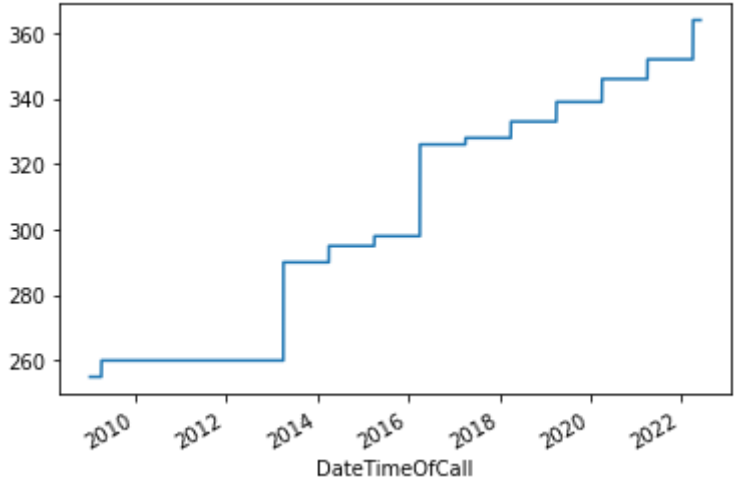
Out[31]:

```
array([255, 260, 290, 295, 298, 326, 328, 333, 339, 346, 352, 3
64])
```

Loading [MathJax]/extensions/Safe.js

In [32]:
```
# just goes up over time
df["HourlyNotionalCost(£)"].plot.line()
```

Out[32]:
```
<AxesSubplot:xlabel='DateTimeOfCall'>
```



Loading [MathJax]/extensions/Safe.js

In [33]:
```python
# Group incidents by fire station & count them
df.groupby("StnGroundName")["IncidentNumber"].count()
```

Out[33]:
```
StnGroundName
Acton          68
Addington      57
Barking        83
Barnet         80
Battersea      70
               ..
Whitechapel    23
Willesden      57
Wimbledon      71
Woodford       83
Woodside       70
Name: IncidentNumber, Length: 107, dtype: int64
```

Loading [MathJax]/extensions/Safe.js

Plot location of calls on a map

- note: this section uses some more libraries, to install them:
- `pip install geopandas contextily`

Loading [MathJax]/extensions/Safe.js

In [34]:

```
# drop missing longitude/latitude
df2 = df.dropna(subset=["Longitude", "Latitude"])
# also drop zero values
df2 = df2[df2["Latitude"] != 0]
# convert to geodataframe using geopandas
import geopandas

# set crs to EPSG:4326 to specify WGS84 Latitude/Longitude
gdf = geopandas.GeoDataFrame(
    df2,
    geometry=geopandas.points_from_xy(df2["Longitude"], df2["Latitude"]
    crs="EPSG:4326",
)
gdf.head()
```

Out[34]:

	IncidentNumber	CalYear	FinYear	PumpCount	PumpHoursTot
DateTimeOfCall					
2009-01-01 08:51:00	275091	2009	2008/09	1.0	1
2009-01-04 10:07:00	2075091	2009	2008/09	1.0	1
2009-01-05 12:27:00	2872091	2009	2008/09	1.0	1

Loading [MathJax]/extensions/Safe.js

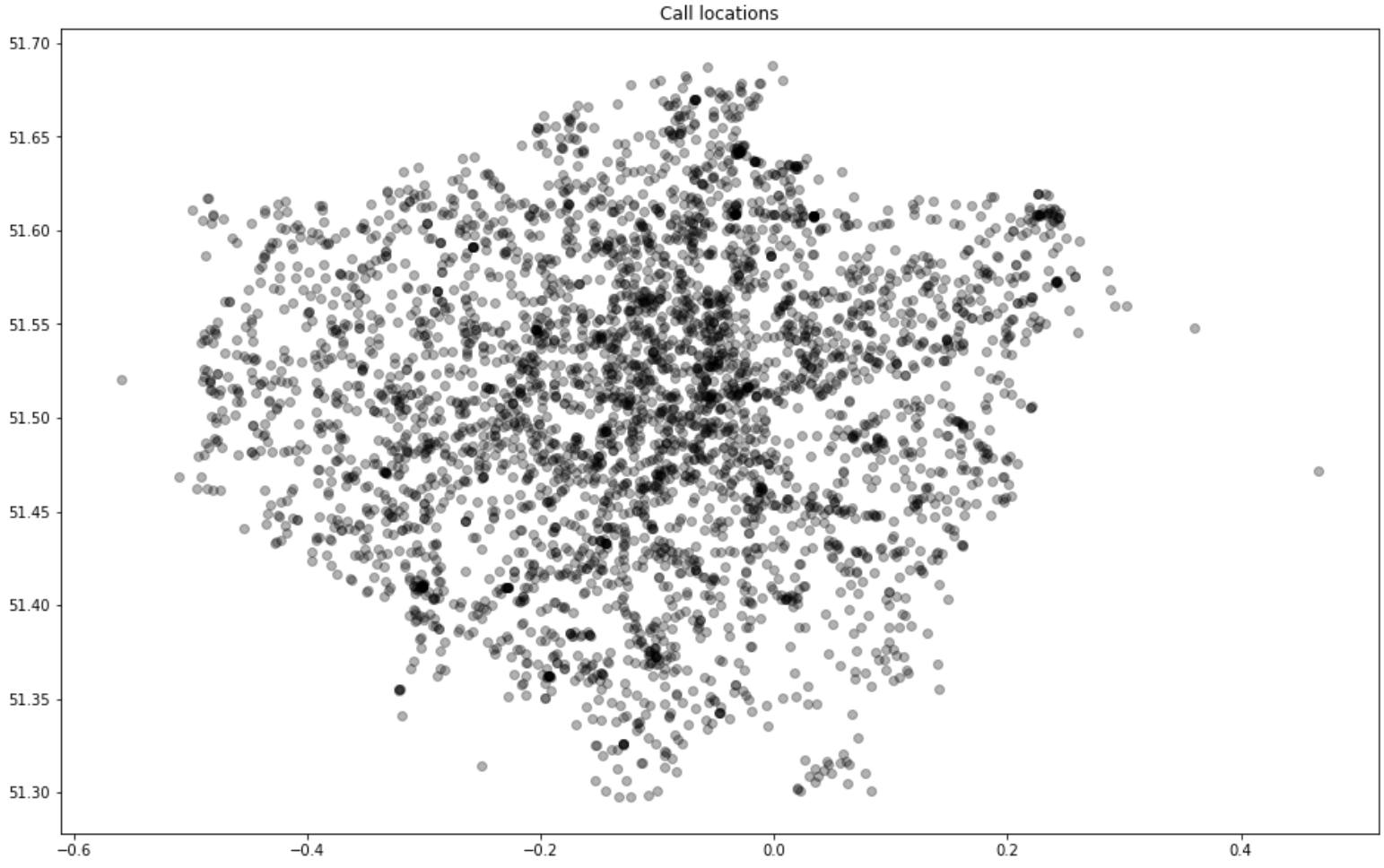
	IncidentNumber	CalYear	FinYear	PumpCount	PumpHoursTot
DateTimeOfCall					
2009-01-07 06:29:00	4011091	2009	2008/09	1.0	1
2009-01-07 11:55:00	4211091	2009	2008/09	1.0	1

5 rows × 30 columns

Loading [MathJax]/extensions/Safe.js

In [35]:

```
f, ax = plt.subplots(figsize=(16, 16))
# plot location of calls involving animals
gdf.plot(ax=ax, color="black", alpha=0.3)
plt.title("Call locations")
# plt.axis("off")
plt.show()
```



Loading [MathJax]/extensions/Safe.js

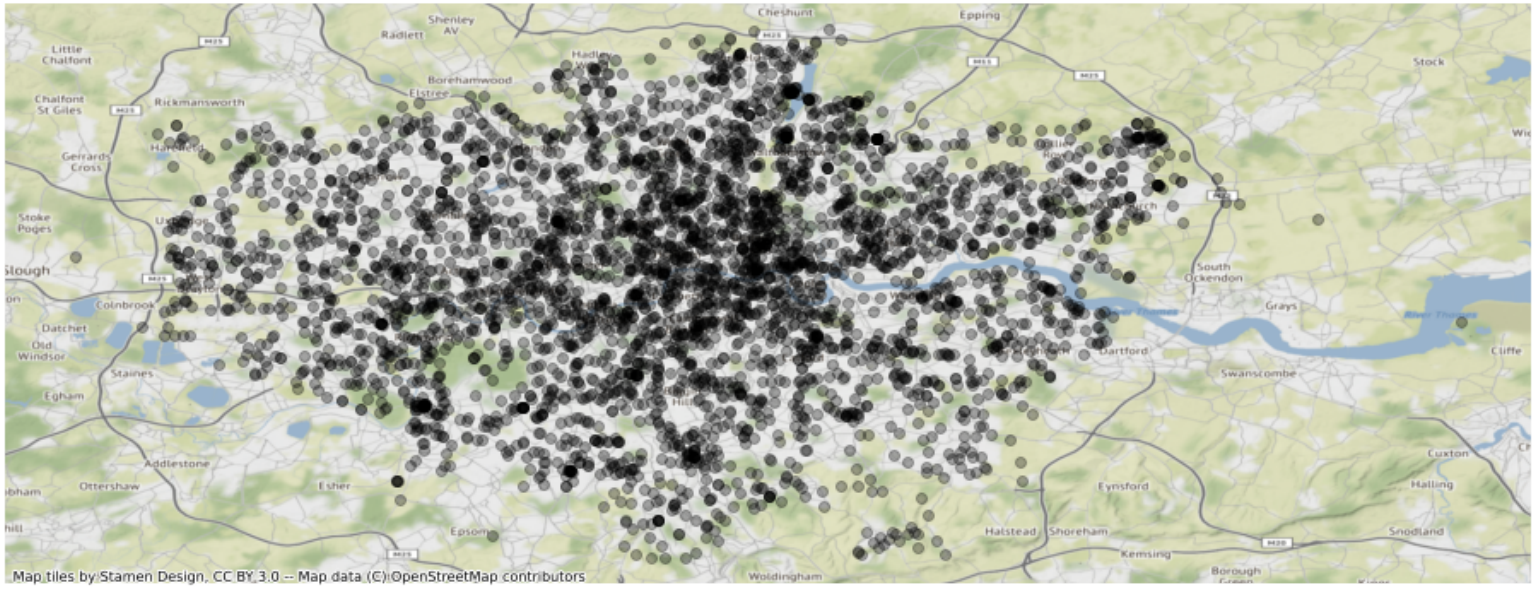
Loading [MathJax]/extensions/Safe.js

In [36]:

```
import contextily as cx

f, ax = plt.subplots(figsize=(16, 16))
# plot location of calls involving animals
gdf.plot(ax=ax, color="black", alpha=0.3)
# add a basemap of the region using contextily
cx.add_basemap(ax, crs=gdf.crs)
plt.title("Call locations")
plt.axis("off")
plt.show()
```

Call locations



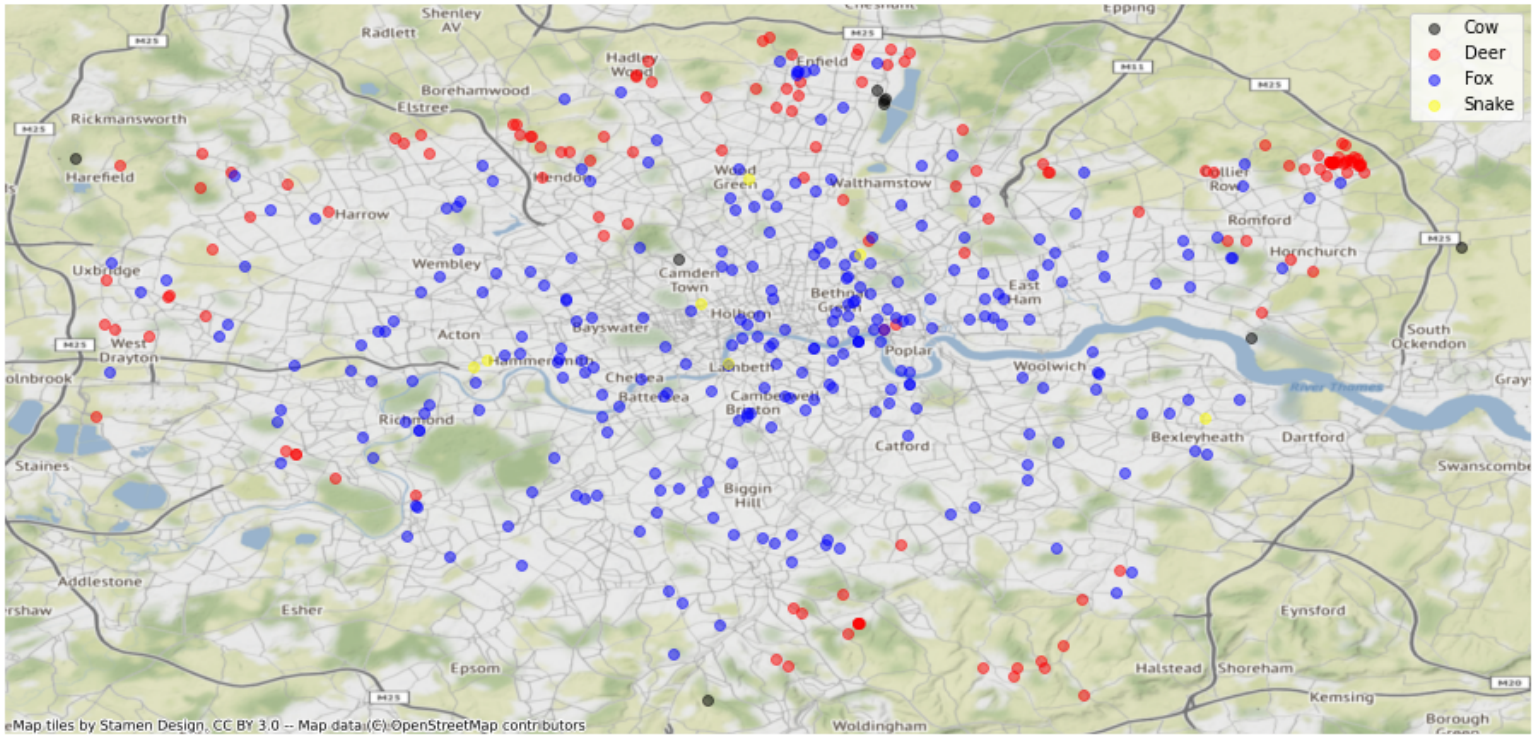
Loading [MathJax]/extensions/Safe.js

In [37]:

```
f, ax = plt.subplots(figsize=(16, 16))
# plot location of calls involving animals
for animal, colour in [
    ("Cow", "black"),
    ("Deer", "red"),
    ("Fox", "blue"),
    ("Snake", "yellow"),
]:
    gdf[gdf["AnimalGroupParent"] == animal].plot(
        ax=ax, color=colour, alpha=0.5, label=animal
    )
# add a basemap of the region using contextily
cx.add_basemap(ax, crs=gdf.crs)
plt.title("Call locations by animal")
plt.legend()
plt.axis("off")
plt.show()
```

Loading [MathJax]/extensions/Safe.js

Call locations by animal



Loading [MathJax]/extensions/Safe.js

Next steps

- experiment with your own datasets
- read some pandas documentation
    - user guide
- follow a tutorial
    - getting started tutorials
- free interactive kaggle courses
    - pandas
    - data cleaning

Loading [MathJax]/extensions/Safe.js